

# Aplicación de los estándares HTML5, CSS3 y JavaScript en el desarrollo de un módulo frontend accesible para el sistema veterinario BIOPET

Johan Stalin Carvajal Loor, Michael Xavier Fajardo Montes, Jaime Josué Mariscal Cabrera Facultad de Ciencias de la Computación  
Universidad Técnica Estatal de Quevedo  
Quevedo, Ecuador

## Resumen

Este trabajo describe el desarrollo del módulo de agendamiento de citas del sistema veterinario BIOPET, construido como una aplicación frontend con HTML5 semántico, CSS3 responsivo y JavaScript modular, sin frameworks adicionales. La aplicación consume la API pública Dog CEO para obtener el catálogo de razas y una imagen referencial de cada paciente, y conserva las citas en el navegador mediante almacenamiento local. El diseño siguió la estrategia *mobile-first* y los principios de accesibilidad de las pautas WCAG 2.1 en su nivel AA. Para verificar la calidad se realizaron auditorías con Lighthouse, WAVE y los validadores del W3C; el resultado final fue de 100 puntos en rendimiento, accesibilidad, buenas prácticas y SEO en Lighthouse, cero errores en WAVE y código HTML y CSS válido según el W3C. Se documenta además la corrección de los dos problemas detectados durante el proceso, lo que evidencia el valor de auditar de forma cuantitativa.

## Index Terms

HTML5, CSS3, JavaScript, accesibilidad web, WCAG 2.1, diseño responsivo, BIOPET.

## I. INTRODUCCIÓN

Las aplicaciones web se han convertido en el medio principal a través del cual las personas acceden a servicios de información, comercio y gestión. En el ámbito de las clínicas veterinarias de pequeña y mediana escala, buena parte de la administración todavía se realiza con herramientas dispersas, como hojas de cálculo o aplicaciones de mensajería, lo que provoca duplicidad de datos y dificulta el seguimiento de la salud de cada mascota. El sistema BIOPET nace para centralizar esa información clínica y administrativa en una sola plataforma accesible desde cualquier dispositivo con conexión a internet.

En esta práctica desarrollamos uno de los módulos de BIOPET, el de agendamiento de citas, como una aplicación del lado del cliente que aplica los estándares del W3C. El propósito no fue replicar el sistema completo, sino ejercitar los fundamentos del desarrollo frontend: estructurar el contenido con HTML5, dar formato y comportamiento responsivo con CSS3, añadir interactividad con JavaScript y consumir una API pública de forma asíncrona. El objetivo concreto fue construir una interfaz semántica, accesible y adaptable a distintos tamaños de pantalla que superara las auditorías de calidad establecidas en la guía de la asignatura: una puntuación de accesibilidad de al menos 90 y de rendimiento de al menos 70 en Lighthouse, además de la conformidad con WCAG 2.1 nivel AA.

El documento se organiza de la manera siguiente. La Sección II presenta el fundamento teórico de la Unidad I. La Sección III describe la implementación del módulo. La Sección IV reúne los resultados de las auditorías de calidad y accesibilidad. Finalmente, la Sección V recoge las conclusiones del trabajo.

## II. FUNDAMENTO TEÓRICO

### II-A. Fundamentos de las aplicaciones web

En el contexto de la ingeniería de software actual, las aplicaciones web se definen como sistemas de software distribuidos que operan sobre la infraestructura de internet, utilizando un navegador convencional como interfaz

de usuario fundamental para interactuar de forma dinámica con los datos e instrucciones de negocio [3]. A lo largo del tiempo, la clasificación de estos entornos ha mutado significativamente; inicialmente divididas bajo una taxonomía simple entre sitios estáticos y dinámicos, las crecientes necesidades del mercado impulsaron la adopción de aplicaciones híbridas que combinaban tecnologías web encapsuladas dentro de capacidades nativas en dispositivos móviles, un enfoque que con frecuencia se enfrentó a serias restricciones de espacio físico en los terminales y a la complejidad técnica de gestionar actualizaciones multiplataforma simultáneas [3]. Como respuesta directa a estas limitaciones operativas, surgieron las Aplicaciones Web Progresivas (PWAs), una categoría arquitectónica intermedia muy potente que aprovecha las APIs estándar de los navegadores modernos para ofrecer una experiencia fluida sumamente cercana a la nativa, fundamentando su confiabilidad e interactividad en componentes estructurales clave como los *service workers*, mecanismos avanzados de almacenamiento local en caché y el modelo de diseño basado en el caparazón de la aplicación o *app shell* [3].

Esta infraestructura tecnológica se sustenta formalmente sobre el paradigma de la arquitectura cliente-servidor, un modelo distribuido en el que las tareas computacionales se reparten entre los proveedores de recursos, denominados servidores, y los demandantes de servicios, conocidos como clientes [4]. En los sistemas web contemporáneos, el cliente ejecuta la interfaz mediante el navegador web y realiza consultas asíncronas de datos hacia endpoints específicos del backend, el cual procesa las solicitudes y devuelve las respuestas estructuradas pertinentes, dinámica que se ha optimizado exponencialmente con el auge de las aplicaciones de una sola página o SPAs [4]. No obstante, la complejidad técnica en el lado del servidor también ha evolucionado de forma drástica para responder a los rigurosos problemas de escalabilidad global y tolerancia a fallos; mientras el enfoque monolítico tradicional concentraba toda la lógica de negocio en un único proceso unificado, las plataformas de alta concurrencia en la industria han impulsado la migración hacia la Arquitectura de Microservicios (MSA) [4]. Este estilo arquitectónico descompone conceptualmente el sistema web en un conjunto de servicios autónomos, modulares y de grano fino organizados en torno a capacidades de negocio, donde cada bloque posee su propio ciclo de vida e implementa patrones descentralizados como el aislamiento de una base de datos por cada servicio individual, reduciendo así el acoplamiento y facilitando un mantenimiento de software iterativo [4].

Para comprender el diseño de estas plataformas es imperativo analizar el viaje histórico de la World Wide Web durante las últimas tres décadas, una línea de tiempo marcada por cambios cuantitativos y cualitativos que transformaron documentos planos en redes inteligentes multimedia de gran escala [5]. Popularmente, este recorrido se divide en tres grandes etapas conceptuales que reflejan la evolución de las herramientas y el rol del usuario [5]. La denominada Web 1.0 representó los cimientos del ecosistema digital, caracterizándose por el uso generalizado de páginas completamente estáticas estructuradas sobre plantillas básicas de HTML donde el usuario asumía una postura estrictamente pasiva de consumo y lectura [5]. El verdadero quiebre tecnológico llegó con la Web 2.0, una fase centrada por completo en la usabilidad, la interoperabilidad y el contenido generado por los propios usuarios; desde la perspectiva de la ingeniería frontend, este salto cualitativo fue viable gracias a la incorporación de la tecnología AJAX y diversos frameworks primitivos de JavaScript que permitieron actualizar de forma dinámica fragmentos específicos de la interfaz sin la necesidad de recargar el documento completo [5]. Finalmente, el avance contemporáneo nos sitúa en la Web 3.0, un enfoque que busca transformar la red global en una gran base de datos distribuida mediante la adición de semántica y metadatos estructurados, permitiendo que las máquinas comprendan la información de manera automatizada para brindar experiencias y búsquedas personalizadas [5].

Este entorno centrado en la web ofrece un conjunto balanceado de ventajas operativas y desventajas críticas que condicionan las decisiones de desarrollo. La ventaja más evidente radica en la compatibilidad multiplataforma inherente y la capacidad de alcanzar a cualquier usuario con una única base de código estándar, eliminando las restricciones de almacenamiento físico y las barreras de aprobación de las tiendas de aplicaciones propietarias [3]. Asimismo, la integración nativa de activos estáticos en caché permite sortear fallas históricas de conectividad al proveer un funcionamiento robusto sin conexión a internet [3]. Sin embargo, este paradigma expone desventajas notables que exigen un estricto control técnico del rendimiento; las actividades de optimización continua demuestran que una velocidad de carga deficiente deteriora críticamente la retención y la satisfacción del usuario en escenarios de uso real, llegando a perjudicar los objetivos comerciales y de conversión ante retrasos de apenas unos pocos milisegundos [6]. A esto se añade la complejidad técnica de gestionar la consistencia de datos, la comunicación de red y las pruebas automatizadas cuando se adoptan sistemas distribuidos [6].

Bajo esta línea de rigurosidad técnica, el diseño web inclusivo y la accesibilidad digital se consolidan como prácticas obligatorias para garantizar que todas las personas puedan percibir, comprender, navegar e interactuar de

forma equitativa con internet, enfocándose de manera prioritaria en eliminar las barreras a las que se enfrentan los usuarios con discapacidades visuales, auditivas, motoras o cognitivas [3], [7]. A nivel internacional, las especificaciones técnicas obligatorias están dictadas por el consorcio W3C a través de las Pautas de Accesibilidad para el Contenido Web, conocidas globalmente como WCAG [7]. La estructura de estas pautas se organiza conceptualmente en torno a cuatro principios fundamentales bajo el acrónimo POUR: perceptible, operable, comprensible y robusto [7]. Durante casi una década, la versión WCAG 2.0 sirvió como el estándar de cumplimiento técnico esencial para portales gubernamentales e institucionales [7]. No obstante, la omnipresencia de los dispositivos móviles obligó a una evolución normativa hacia la versión WCAG 2.1 [7]. Esta actualización no anula los criterios previos, sino que añade diecisiete nuevos criterios de éxito orientados específicamente a optimizar la accesibilidad en interfaces táctiles, de baja visión y de soporte cognitivo, adaptando el espaciado del texto y simplificando los flujos de navegación para mitigar la sobrecarga cognitiva [7]. La integración nativa de estas pautas dentro de los sistemas de diseño unificados garantiza que la inclusión sea un componente estructural fundacional y no un parche correctivo de última hora [7].

## *II-B. HTML y HTML5 para el desarrollo web*

En los orígenes del desarrollo web, el lenguaje HTML (HyperText Markup Language) se consolidó como el estándar de marcado de hipertexto universal diseñado para definir la estructura básica y semántica de las páginas mediante el uso de etiquetas [9]. Durante las primeras fases evolutivas de la red, la presentación de los sitios dependía enteramente de este marcado básico, lo que forzaba a los desarrolladores a entremezclar elementos estructurales con estilos de formato tipográfico mediante etiquetas obsoletas como `<FONT>`, `<CENTER>` o `<B>`, dando como resultado plataformas de difícil mantenimiento y con serias barreras de acceso universal [10]. Esta evolución técnica comparte bases de diseño comunes con otras familias de lenguajes de marcado estructurado como XML y la transferencia de datos tipificados por esquemas MIME [9], [10]. Al procesar estos documentos, los intérpretes de los navegadores ejecutan el código de forma secuencial; cuando el motor se encuentra con una de estas etiquetas de script, suspende de manera inmediata la renderización visual de la página para conmutar al modo JavaScript y ejecutar las instrucciones, regresando al modo HTML una vez finalizada la interpretación del script [8].

Con el advenimiento del estándar HTML5, cuya especificación oficial fue presentada el 28 de octubre de 2014 por el W3C, se integraron elementos de marcado semántico nativo como `<header>`, `<nav>`, `<main>`, `<section>`, `<article>`, `<aside>` y `<footer>` [9]. La implementación de esta estructura con significado semántico no solo potencia significativamente el posicionamiento en motores de búsqueda (SEO) y facilita la legibilidad del código, sino que resulta fundamental para la accesibilidad web, al proveer marcas de navegación claras para los lectores de pantalla y tecnologías de apoyo [9]. Asimismo, HTML5 transformó los formularios web introduciendo validaciones nativas y tipos de campo especializados como correos, urls o fechas, y eliminó la necesidad histórica de complementos propietarios de terceros para multimedia gracias al soporte directo de las etiquetas `<audio>` y `<video>` [9]. Este ecosistema se complementa con potentes capacidades de desarrollo a través de APIs de almacenamiento local (*Local Storage*), geolocalización, hilos de ejecución con Web Workers, y el elemento `<canvas>` para el renderizado programático de gráficos e ilustraciones bidimensionales interactivas en tiempo real [9].

## *II-C. CSS para el diseño y la presentación web*

Los fundamentos de las Hojas de Estilo en Cascada (CSS) se asientan sobre la separación estricta entre el contenido estructural y su presentación visual, utilizando en la actualidad una rica taxonomía de selectores avanzados, pseudoclasas estructurales como `nth-child` y pseudoelementos como `::before` o `::after` que elevan la especificidad y evitan la proliferación de clases redundantes en el marcado [11]. Toda maquetación tradicional web se ha estructurado históricamente sobre el modelo de caja, el cual define áreas concéntricas de contenido, relleno (*padding*), borde y margen, implementando flujos de diseño líquido mediante el posicionamiento de bloques flotantes (`float: left`) y anchos expresados en porcentajes para impedir la aparición de barras de desplazamiento horizontal [13]. Con la maduración del estándar hacia CSS3, la maquetación dio un salto cualitativo mediante la introducción de dos sistemas modernos complementarios: Flexbox, un modelo unidimensional diseñado para la distribución eficiente del espacio y alineación de componentes dinámicos en un solo eje, y CSS Grid Layout, un

sistema bidimensional idóneo para controlar simultáneamente filas y columnas a nivel macroestructural mediante unidades fraccionales ( $\text{fr}$ ) [11], [13].

Frente a la disyuntiva clásica de elegir un único modelo, las mejores prácticas contemporáneas promueven la creación de maquetaciones híbridas, donde Grid asume el andamiaje estructural externo del sitio y Flexbox gobierna la flexibilidad del contenido interno dentro de las celdas de la cuadrícula [15]. Paralelamente, el Diseño Web Responsivo (RWD), teorizado por Ethan Marcotte en 2010, establece que el contenido debe adaptarse fluidamente a múltiples resoluciones y orientaciones mediante rejillas elásticas y consultas de medios (*media queries*) que segmentan la visualización en puntos de ruptura o *breakpoints* fijos [12], [13]. Esta filosofía se materializa a través de la estrategia *mobile-first*, un enfoque metodológico donde se diseñan primero los estilos base para teléfonos inteligentes —los cuales operan bajo severas restricciones como pantallas que representan el 25 % del tamaño de un monitor de escritorio, memorias RAM limitadas de 256MB a 2GB, anchos de banda reducidos e interacciones puramente táctiles en lugar de eventos de cursor (*hover*)— para luego añadir de forma progresiva consultas de medios aplicables a pantallas mayores [12]. En este proceso de adaptación fluida, se emplean fórmulas de conversión proporcional del tipo “objetivo % contexto = resultado” para transformar anchos fijos en porcentajes elásticos [12], [14].

Para evaluar las limitantes inherentes del CSS puro, tales como la redundancia de código y la ausencia de lógica computacional, se recurre a preprocesadores CSS como SASS (2007), LESS (2008) y Stylus (2010), los cuales actúan como metalenguajes que incorporan variables dinámicas (ej. `$alert_color`), selectores anidados con el operador `&` y directivas `@extend` bajo el principio DRY (*Don't Repeat Yourself*) [14]. La mayor ventaja técnica de este enfoque radica en la creación de *mixins* reutilizables (mediante `@mixin` y `@include`) que automatizan las maquetaciones híbridas, logrando reducir hasta un 30 % los estilos duplicados y acelerando los cálculos de reflujo (*reflows*) en un 25 % durante los ciclos de mantenimiento del software, todo ello estructurado de forma limpia bajo arquitecturas modulares como BEM, SMACSS o ITCSS [11], [14], [15].

## II-D. JavaScript para aplicaciones web

Como motor esencial de la interactividad en el entorno frontend, JavaScript se define como un lenguaje de programación dinámico, multiplataforma, orientado a objetos y de alto rendimiento integrado de forma nativa en los navegadores web modernos [8]. En sus fundamentos, este lenguaje destaca por operar mediante un motor de interpretación embebido que traduce el código fuente en instrucciones de máquina “as is” en tiempo real, optimizando dinámicamente la velocidad de respuesta de las interfaces de usuario mediante compilación JIT [8]. El manejo de la información dentro del script se articula a través de variables tipificadas, cuya definición evolucionó desde el uso tradicional de la palabra clave `var` hacia los contenedores modernos de ámbito de bloque `let` y `const`, proporcionando un control de alcance mucho más restrictivo y reduciendo errores comunes de reasignación y elevación (*hoisting*) en tiempo de ejecución [18]. Esta base se complementa con tipos de datos primitivos (como cadenas, números y booleanos) junto con estructuras compuestas, permitiendo que el software gestione dinámicamente datos estructurados para responder de inmediato a los requerimientos del usuario [16].

Para organizar la lógica de negocio en el lado del cliente, el lenguaje se apoya en estructuras de control secuenciales, condicionales y cíclicas, las cuales gobiernan el flujo de ejecución basándose en evaluaciones lógicas mediante bloques de decisión `if/else`, construcciones selectivas `switch` y bucles iterativos estructurados como `for`, `while` y `do/while` [18], [16]. No obstante, el pilar conceptual fundamental que otorga modularidad al desarrollo es la concepción de las funciones como ciudadanos de primera clase, lo que significa que pueden ser asignadas libremente a variables, pasadas como argumentos funcionales a otras rutinas (*callbacks*) o retornadas como valores desde bloques de ejecución autónomos [16]. Este paradigma facilita el encapsulamiento de código reutilizable y sienta las bases técnicas para el diseño modular, un aspecto crítico al coordinar flujos algorítmicos complejos sin sobrecargar el hilo principal de ejecución del sistema frontend [18].

La comunicación directa entre la lógica algorítmica y la representación visual se realiza mediante la interfaz de programación del Modelo de Objetos del Documento, conocida como la API del DOM [18]. Este mecanismo expone el árbol estructurado del documento HTML como un conjunto jerárquico de nodos interactivos que JavaScript puede interrogar, modificar, inyectar o eliminar en respuesta a los estímulos externos [17]. Dicha interacción está estrictamente gobernada por un paradigma basado en eventos, donde el desarrollador establece escuchadores especializados mediante el método `addEventListener` vinculados a elementos específicos de la interfaz [17].

Cuando se dispara una acción del usuario —como un clic, el envío de un formulario, la carga de activos de red o la pulsación de una tecla—, el sistema captura el objeto del evento y ejecuta la función manejadora asignada, permitiendo alterar el árbol de renderizado o los estilos CSS sin forzar una recarga total del navegador [17].

Por último, la maduración del estándar impulsada por la especificación ECMAScript 6 (ES6+) y versiones subsiguientes transformó profundamente el desarrollo frontend al introducir una sintaxis más limpia y expresiva a través de funciones de flecha, desestructuración de objetos, literales de plantilla y módulos nativos, al tiempo que revolucionó las arquitecturas asíncronas [16], [17]. Históricamente, la gestión de operaciones costosas en tiempo, como el consumo de APIs externas o peticiones de red Ajax, dependía de múltiples funciones de devolución anidadas que degradaban críticamente la mantenibilidad en un fenómeno conocido como el infierno de los callbacks o *callback hell* [17]. La introducción de las Promesas nativas y, posteriormente, de la sintaxis moderna basada en operadores `async/await` resolvió esta problemática estructural al permitir redactar flujos asíncronos con la apariencia y simplicidad de un código secuencial lineal [17]. Este avance resulta crucial para aplicaciones contemporáneas como el módulo de agendamiento de BIOPET, garantizando que el consumo asíncrono de recursos con la API `fetch` ocurra en segundo plano de manera no bloqueante, protegiendo así el rendimiento de la interfaz gráfica [8], [17].

### III. DESARROLLO DEL MÓDULO

El módulo se implementó como una aplicación frontend independiente, sin frameworks ni dependencias externas, con el fin de trabajar directamente sobre los fundamentos de HTML, CSS y JavaScript. Conviene aclarar que esta aplicación corresponde al ejercicio de la Unidad I; el sistema BIOPET completo, con su frontend en Angular y su API en ASP.NET Core, se desarrolla de forma separada a lo largo del semestre.

El entorno de trabajo se basó en Visual Studio Code con la extensión Live Server para la recarga automática, Git y GitHub para el control de versiones, y Google Chrome para las pruebas. El código se organizó en tres archivos principales: `index.html` para la estructura, `styles.css` para la presentación y una carpeta `js` con tres módulos de JavaScript.

El historial de commits del repositorio evidencia la participación de los integrantes del equipo en el desarrollo, corrección y documentación del módulo, como se muestra en el Anexo C.

#### III-A. Estructura semántica y formulario

La página se construyó con los elementos semánticos de HTML5 y con atributos de accesibilidad (ARIA) donde fueron necesarios. El formulario de la cita reúne más de ocho tipos distintos de campo —texto, número, correo, teléfono, fecha, hora, rango, archivo, selección, opción única, casilla y área de texto— cada uno con su etiqueta asociada, validación nativa y mensajes de error que se anuncian a los lectores de pantalla. La Fig. 1 muestra, de forma resumida, la estructura del documento.

Además, las tarjetas dinámicas de las citas agendadas se renderizan como elementos `<article>`, ya que cada cita representa una unidad independiente de contenido dentro del módulo. De esta manera, la estructura del documento evidencia el uso de `<header>`, `<nav>`, `<main>`, `<section>`, `<article>`, `<aside>` y `<footer>`, conforme a los criterios semánticos establecidos en la guía.

#### III-B. Diseño responsivo

Los estilos se escribieron siguiendo la estrategia *mobile-first*: primero se definieron los estilos base para pantallas pequeñas y luego, mediante *media queries*, se añadieron los ajustes para pantallas mayores. El sistema se apoya en variables CSS que centralizan los colores y los espacios, lo que facilita mantener la coherencia visual y, sobre todo, implementar un modo oscuro. La disposición general de la página utiliza CSS Grid, mientras que los componentes internos, como la barra de navegación o las tarjetas, se alinean con Flexbox. El modo oscuro se activa automáticamente según la preferencia del sistema operativo y, además, puede alternarse de forma manual; esa elección se guarda en el navegador. La Fig. 2 resume estas ideas.

```

1 <header>
2   <nav aria-label="Navegación principal">...</nav>
3 </header>
4 <main id="contenido">
5   <section aria-labelledby="agendar-titulo">
6     <h2 id="agendar-titulo">Agendar cita</h2>
7     <form id="form-cita" novalidate>...</form>
8   </section>
9   <section aria-labelledby="citas-titulo">...</section>
10  <aside>...</aside>
11 </main>
12 <footer>...</footer>
13 <dialog id="dialogo-detalle"
14   aria-labelledby="dialogo-titulo">...</dialog>

```

Figura 1. Estructura semántica resumida del documento.

```

1 :root {
2   --c-brand: #0e7d72;
3   --espaciado-md: 1rem;
4   /* ... más tokens de diseño ... */
5 }
6 @media (prefers-color-scheme: dark) {
7   :root:not([data-tema="claro"]) {
8     --c-brand: #38c0b0; /* ... */
9   }
10 }
11 .disposicion { display: grid; grid-template-columns: 1fr; }
12 @media (min-width: 920px) {
13   .disposicion { grid-template-columns: 2.2fr 1fr; }
14 }
15 .nav { display: flex; gap: var(--espaciado-md); }

```

Figura 2. Variables, disposición con Grid/Flexbox y modo oscuro.

### III-C. Interactividad y consumo de la API

La capa de JavaScript se dividió en tres módulos con responsabilidades separadas: `api.js` se encarga de comunicarse con el servidor, `ui.js` de manipular el DOM y mostrar los distintos estados de la interfaz, y `app.js` de coordinar ambos y gestionar los eventos. Esta separación facilita entender y mantener el código.

La aplicación consume la API pública Dog CEO de dos maneras. Al cargar la página obtiene el catálogo de razas y lo coloca en el campo de selección correspondiente; si la petición falla, se utiliza una lista básica de respaldo y se muestra un aviso con la opción de reintentar. Al agendar una cita, la aplicación solicita una imagen referencial de la raza elegida para mostrarla en la tarjeta. Toda la comunicación se realiza con `fetch` y `async/await`, comprobando de forma explícita el estado de la respuesta, ya que `fetch` no rechaza la promesa ante errores del servidor. La Fig. 3 muestra la función que obtiene la imagen.

Además del consumo de la API, el módulo incorpora una búsqueda en vivo que filtra las citas a medida que se escribe, un cuadro de diálogo accesible que muestra el detalle de cada cita gestionando el foco del teclado, y la posibilidad de cancelar una cita. Las citas agendadas se conservan en el almacenamiento local del navegador, de modo que permanecen disponibles aunque se recargue la página.

## IV. RESULTADOS: AUDITORÍA DE CALIDAD Y ACCESIBILIDAD

Para evaluar la calidad del módulo se emplearon tres herramientas complementarias, tal como recomienda la guía de la asignatura, ya que ninguna detecta por sí sola todos los problemas. Lighthouse, integrado en las herramientas de desarrollo de Chrome, entrega puntuaciones automáticas en rendimiento, accesibilidad, buenas prácticas y SEO. WAVE señala sobre la propia página los problemas de accesibilidad y contraste. Por último, los validadores del W3C comprueban que el código HTML y CSS cumpla la especificación.

Las auditorías se realizaron en dos momentos: una primera medición y otra después de corregir los problemas encontrados, siguiendo el enfoque de comparación “antes y después”. En la primera ejecución, Lighthouse otorgó 100 puntos en rendimiento, buenas prácticas y SEO, y 96 en accesibilidad. El único inconveniente fue un contraste

```

1 // api.js - consumo de la API pública Dog CEO
2 const BASE_URL = 'https://dog.ceo/api';
3
4 export async function obtenerImagenRaza(raza, signal) {
5   const url = `${BASE_URL}/breed/${raza}/images/random`;
6   const r = await fetch(url, { signal });
7   if (!r.ok) {
8     throw new Error(`Error HTTP ${r.status}`);
9   }
10  const datos = await r.json();
11  return datos.message; // URL de la imagen
12 }

```

Figura 3. Petición asíncrona a la API con manejo de errores.

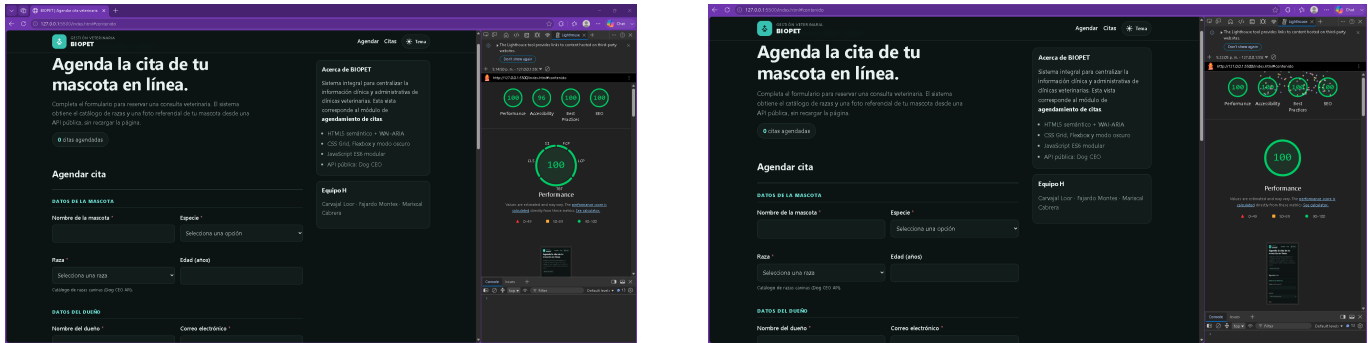


Figura 4. Auditoría Lighthouse antes (izquierda, accesibilidad 96) y después (derecha, accesibilidad 100) de corregir el contraste del botón principal.

insuficiente entre el texto blanco y el fondo del botón principal cuando la interfaz estaba en modo oscuro. Tras ajustar el color del texto de ese botón, la puntuación de accesibilidad subió a 100 (Fig. 4).

En WAVE, el módulo no presentó errores ni problemas de contraste, y obtuvo la máxima calificación de la herramienta (Fig. 5). La validación de HTML del W3C reportó inicialmente un error, causado por un elemento output ubicado dentro de una etiqueta label cuyo atributo apuntaba a otro control, lo que generaba ambigüedad; al reemplazar ese elemento por uno no etiquetable, la validación quedó sin errores. La validación de CSS no presentó errores en ningún momento. La Tabla I resume todos los valores.

El contraste mínimo se verificó sobre el botón principal en modo oscuro, debido a que fue el elemento observado inicialmente por Lighthouse. El color del texto `rgb(6, 35, 31)` sobre el fondo `rgb(56, 192, 176)` obtuvo un ratio aproximado de 7.36:1, superando el mínimo 4.5:1 requerido por WCAG 2.1 nivel AA para texto normal.

La validez del código quedó documentada con las capturas de los validadores del W3C (Fig. 6). En conjunto, los resultados muestran que el módulo cumple con holgura los objetivos planteados y se ajusta al nivel AA de las pautas WCAG 2.1 [2].

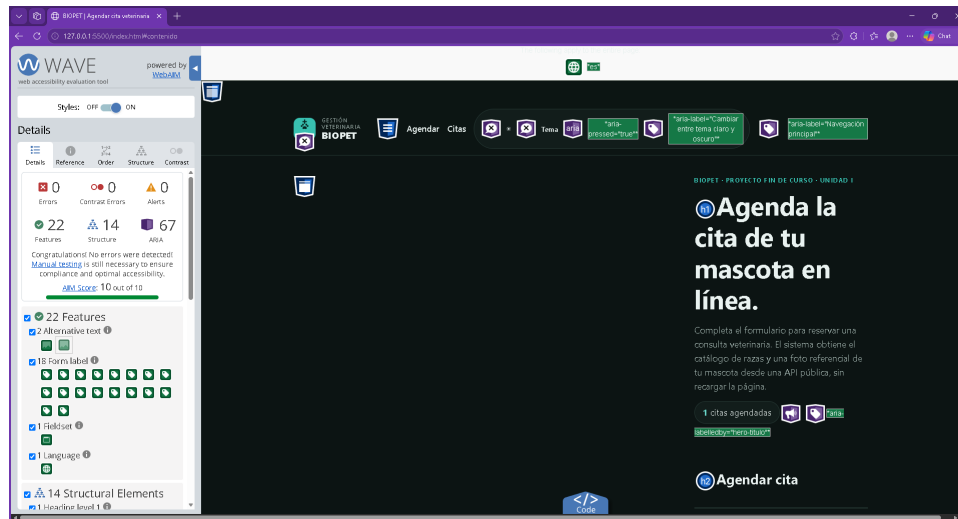


Figura 5. Resumen de WAVE: cero errores, cero errores de contraste y cero alertas.

Tabla I  
MÉTRICAS DE CALIDAD: ANTES Y DESPUÉS DE LAS CORRECCIONES

| Métrica                       | Antes     | Después  | Objetivo     |
|-------------------------------|-----------|----------|--------------|
| Lighthouse — Accesibilidad    | 96        | 100      | $\geq 90$    |
| Lighthouse — Rendimiento      | 100       | 100      | $\geq 70$    |
| Lighthouse — Buenas prácticas | 100       | 100      | $\geq 90$    |
| Lighthouse — SEO              | 100       | 100      | —            |
| WAVE — Errores                | 0         | 0        | 0            |
| WAVE — Alertas                | 0         | 0        | $< 5$        |
| Errores W3C HTML              | 1         | 0        | 0            |
| Errores W3C CSS               | 0         | 0        | 0            |
| Ratio de contraste mínimo     | $< 4,5:1$ | $7,36:1$ | $\geq 4,5:1$ |

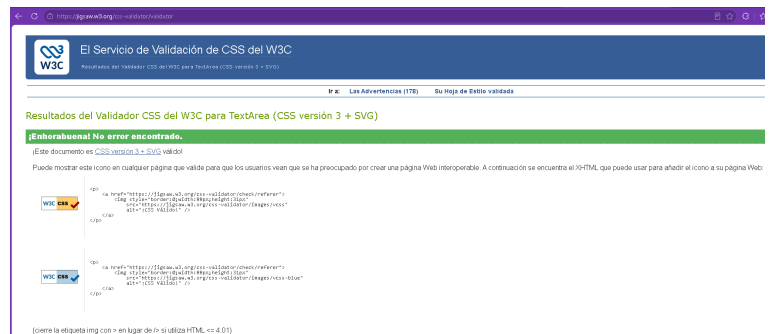
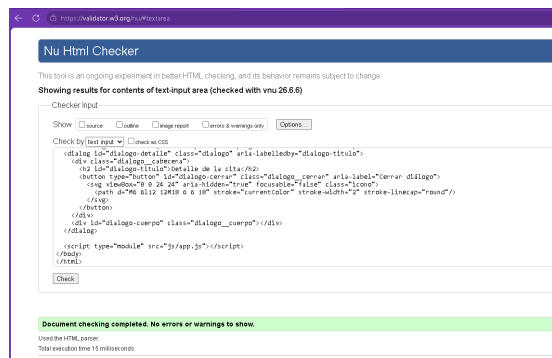


Figura 6. Validadores del W3C: HTML sin errores (izquierda) y CSS sin errores (derecha).



## V. CONCLUSIONES

El desarrollo del módulo de agendamiento de citas permitió aplicar de forma concreta los estándares estudiados en la Unidad I. La estructura semántica de HTML5, además de organizar mejor el contenido, resultó clave para la accesibilidad y el posicionamiento, tal como plantea la literatura revisada. El uso de CSS Grid para la disposición general y de Flexbox para los componentes, junto con la estrategia *mobile-first* y las variables de diseño, hizo posible una interfaz responsiva y un modo oscuro sin recurrir a herramientas externas. Por su parte, la organización del JavaScript en módulos con responsabilidades separadas y el consumo asíncrono de la API mostraron cómo dotar de interactividad a una página manteniendo un código claro y manejable.

El proceso de auditoría confirmó que la calidad de una aplicación web debe verificarse de manera cuantitativa y con varias herramientas a la vez. Las dos correcciones aplicadas —el contraste del botón en modo oscuro y un elemento mal anidado en el formulario— fueron pequeñas, pero sin las auditorías habrían pasado inadvertidas. El resultado final, con la máxima puntuación en Lighthouse, ausencia de errores en WAVE y código válido según el W3C, demuestra que es posible alcanzar los objetivos de accesibilidad y rendimiento desde la etapa de fundamentos, y sienta una base sólida para las siguientes fases del proyecto BIOPET.

## REFERENCIAS

- [1] R. Tabarés, “HTML5 and the evolution of HTML; tracing the origins of digital platforms,” *Technology in Society*, vol. 65, art. 101529, 2021.
- [2] World Wide Web Consortium (W3C), “Web Content Accessibility Guidelines (WCAG) 2.1,” W3C Recommendation, 2018. [En línea]. Disponible: <https://www.w3.org/TR/WCAG21/>
- [3] K. I. Roumeliotis and N. D. Tselikas, “Evaluating Progressive Web App Accessibility for People with Disabilities,” *Network*, vol. 2, no. 2, pp. 350–369, 2022.
- [4] M. Söylemez, B. Tekinerdogan, and A. K. Tarhan, “Challenges and Solution Directions of Microservice Architectures: A Systematic Literature Review,” *Applied Sciences*, vol. 12, no. 11, p. 5507, 2022.
- [5] V. Agarwal and N. Sastry, ““Way back then”: A Data-driven View of 25+ years of Web Evolution,” in *Proceedings of the ACM Web Conference 2022*, pp. 3471–3479, 2022.
- [6] J. van Riet, I. Malavolta, and T. A. Ghaleb, “Optimize along the way: An industrial case study on web performance,” *The Journal of Systems & Software*, vol. 198, p. 111593, 2023.
- [7] H. Shah, “Advancing Web Accessibility: A Guide to Transitioning Design Systems from WCAG 2.0 to WCAG 2.1,” *Computer Science & Information Technology (CS & IT)*, pp. 233–245, 2023.
- [8] M. Shoikhedbrod, “Using JavaScript in web design,” *BOHR International Journal of Engineering*, vol. 2, no. 1, pp. 7–14, 2023.
- [9] “Fundamentos de las Aplicaciones Web,” *Investigación Académica*, pp. 1–20, 2024.
- [10] “CSS para el Diseño y la Presentación Web: Fundamentos, Técnicas Modernas y Mejores Prácticas,” *Informe Académico*, 2026.
- [11] S. Aravind, H. Cherukuri, R. K. Gupta, S. Shukla, and A. T. Rajan, “The Role of HTML5 and CSS3 in Creating Optimized Graphic Prototype Websites and Application Interfaces,” *NeuroQuantology*, vol. 20, no. 12, pp. 4522–4536, 2022.
- [12] B. Prakash, “Retrofitting Mobile First Design, Responsive Design: Driving Factors, Approach, Best Practices and Design Considerations,” *Current Trends in Computer Sciences & Applications*, vol. 2, no. 2, pp. 167–178, 2020.
- [13] N. Bhanarkar, A. Paul, and A. Mehta, “Responsive Web Design and Its Impact on User Experience,” *International Journal of Advanced Research in Science, Communication and Technology (IJARSCT)*, vol. 3, no. 4, pp. 50–55, 2023.
- [14] A. A. Beliaev, “Preprotsessory CSS v praktike veb-dizayna: vozmozhnosti i perspektivy” (Preprocesadores CSS en la práctica del diseño web: posibilidades y perspectivas), *Vestnik Moskovskogo Universiteta*, 2026.
- [15] K. Muppaneni, “Hybrid Layouts: Grid + Flex + Mixins,” *American International Journal of Computer Science and Technology (AIJCST)*, vol. 5, no. 2, pp. 23–35, 2023.
- [16] K. Simpson, *You Don’t Know JS: Up & Going*. Sebastopol, CA: O’Reilly Media, 2015.
- [17] R. Sekuloski, *JavaScript From Zero to Hero: The Most Complete Guide Ever*. Australia: Independently Published, 2022.
- [18] GoalKicker.com, *JavaScript Notes for Professionals*. Free Programming Books, 2018.

## ANEXO A

### CAPTURAS DE PANTALLA DE LA INTERFAZ DE USUARIO (BIOPET)

En este apartado se presentan las evidencias visuales del comportamiento funcional del módulo frontend desarrollado para el agendamiento de citas del sistema veterinario BIOPET. Las capturas demuestran la adaptabilidad del diseño bajo la estrategia *mobile-first* y la consistencia en la distribución del espacio utilizando maquetaciones híbridas basadas en CSS Grid y Flexbox. Asimismo, se exhibe la correcta implementación de la inversión cromática automatizada y manual correspondiente al modo claro y modo oscuro, salvaguardando los contrastes de color exigidos por las pautas de accesibilidad WCAG 2.1 nivel AA.

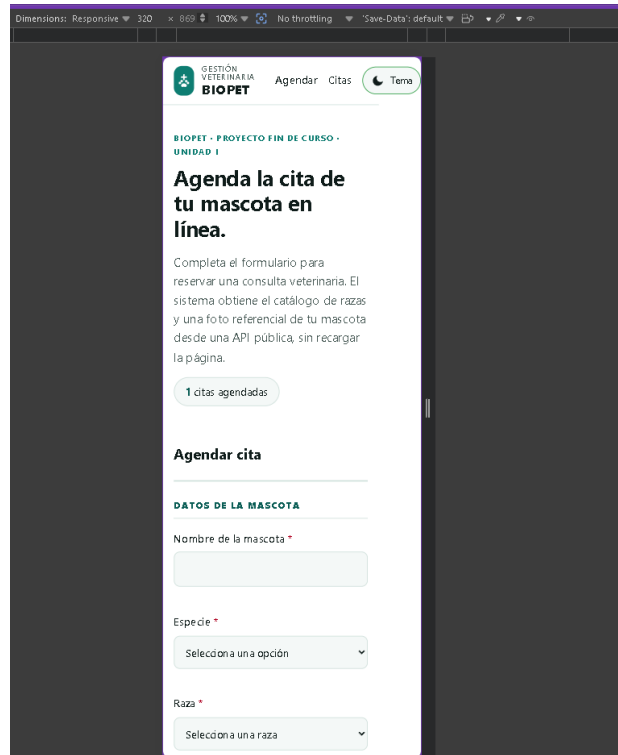


Figura 7. Vista optimizada del formulario de agendamiento en dispositivos móviles con validación nativa activada.

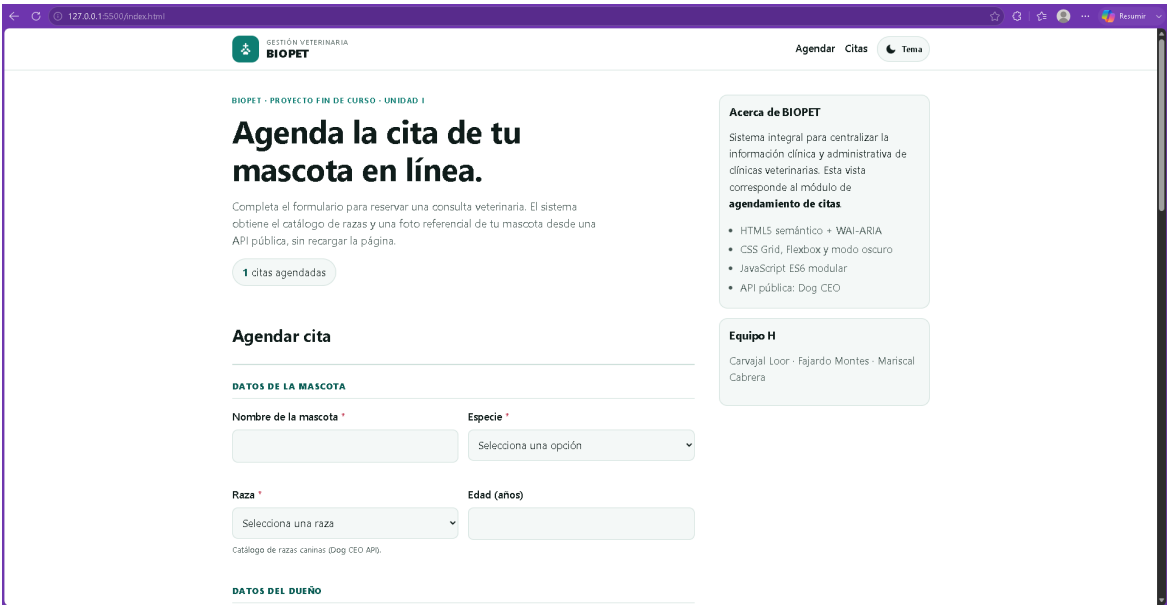


Figura 8. Disposición general de la aplicación en pantallas de escritorio bajo el tema visual claro, mostrando la barra de navegación semántica y el panel de citas.

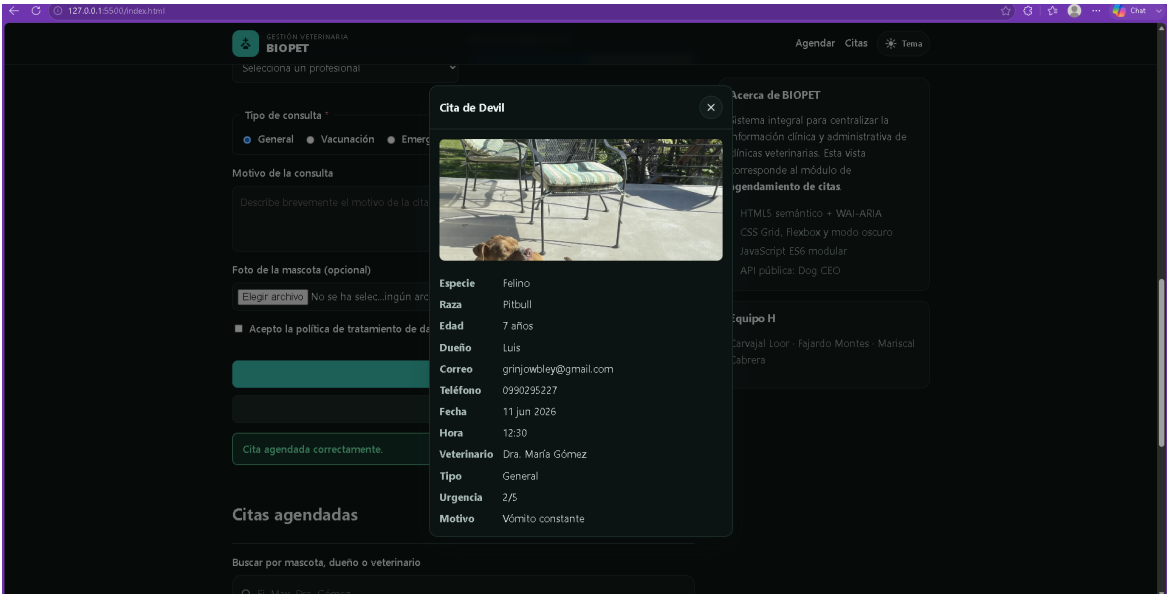


Figura 9. Interfaz completa en entorno de escritorio reflejando la persistencia del modo oscuro y la inyección dinámica de la imagen referencial obtenida desde la API pública Dog CEO.

## ANEXO B

### FRAGMENTOS DEL CÓDIGO FUENTE Y ARQUITECTURA MODULAR

A continuación, se documentan los bloques de código fuente esenciales que gobiernan la lógica síncrona y asíncrona del lado del cliente. Los fragmentos seleccionados ilustran la implementación de la manipulación del árbol del DOM sin dependencias de librerías externas, la captura optimizada de eventos del usuario mediante escuchadores periféricos y el control del flujo de ejecución asíncrono utilizando los operadores modernos `async/await` para gestionar las promesas de red de forma no bloqueante.



```

js > JS api.js > ...
6
7   const BASE_URL = 'https://dog.ceo/api';
8
9   /**
10    * Obtiene el catálogo de razas caninas.
11    * @param {AbortSignal} [signal] - Señal para cancelar la petición.
12    * @returns {Promise<string[]>} Arreglo de razas (claves en minúscula), ordenado.
13    * @throws {Error} Si la respuesta HTTP no es satisfactoria.
14    */
15   export async function obtenerRazas(signal) {
16     const respuesta = await fetch(`${BASE_URL}/breeds/list/all`, { signal });
17
18     // fetch NO rechaza ante 4xx/5xx: se comprueba response.ok de forma explícita.
19     if (!respuesta.ok) {
20       throw new Error(`Error HTTP ${respuesta.status} (${respuesta.statusText})`);
21     }
22     const datos = await respuesta.json();
23     if (datos.status !== 'success') {
24       throw new Error('La API devolvió una respuesta no válida.');
```

Figura 10. Captura del módulo `api.js` evidenciando el control explícito sobre los códigos de estado HTTP al consumir recursos externos.

```

Welcome  index.html  # styles.css  JS api.js  JS ui.js  JS app.js
js > JS ui.js > alternarTema
6
7   const refs = {
8     lista: document.getElementById('lista-citas'),
9     cargando: document.getElementById('estado-cargando'),
10    cargandoTexto: document.getElementById('estado-cargando-texto'),
11    vacio: document.getElementById('estado-vacio'),
12    error: document.getElementById('estado-error'),
13    errorTexto: document.getElementById('estado-error-texto'),
14    conteo: document.getElementById('conteo'),
15    raza: document.getElementById('raza'),
16    dialogo: document.getElementById('dialogo-detalle'),
17    dialogoTitulo: document.getElementById('dialogo-titulo'),
18    dialogoCuerpo: document.getElementById('dialogo-cuerpo'),
19    formMensaje: document.getElementById('form-mensaje'),
20    btnTema: document.getElementById('btn-tema'),
21  };
22
23  const CLAVE_TEMA = 'biopet-tema'; // clave de Web Storage (localStorage)
24
25  // --- Utilidades ---
26  function capitalizar(texto) {
27    return texto.charAt(0).toUpperCase() + texto.slice(1);
28  }
29
30  function formatearFecha(iso) {
31    // iso en formato yyyy-mm-dd
32    const [a, m, d] = iso.split('-');
33    const fecha = new Date(Number(a), Number(m) - 1, Number(d));
34    return fecha.toLocaleDateString('es-EC', { day: '2-digit', month: 'short', year: 'numeric' });
35  }
36
37  function claseBadge(tipo) {
38    if (tipo === 'Emergencia') return 'badge--emergencia';
39    if (tipo === 'Vacunación') return 'badge--vacunacion';
40    return 'badge--general';
41  }
42
43  // --- Catálogo de razas (poblar el <select>) ---
44  export function poblarRazas(razas) {
45    refs.raza.innerHTML = '';
46    const placeholder = new Option('Selecciona una raza', '');
47    placeholder.disabled = true;
48    placeholder.selected = true;
49    refs.raza.appendChild(placeholder);
50    for (const r of razas) {
51      refs.raza.appendChild(new Option(capitalizar(r), r));
52    }
53  }
54

```

Figura 11. Fragmento técnico del módulo `ui.js` encargado del renderizado dinámico de los componentes e inyección de plantillas de marcado.

## ANEXO C

### EVIDENCIA DEL REPOSITORIO GITHUB Y COMMITS DEL EQUIPO

Como parte del control de versiones del proyecto, el equipo utilizó un repositorio compartido en GitHub para registrar los avances realizados durante el desarrollo del módulo frontend de BIOPET. La Fig. 12 evidencia el historial de commits del repositorio, donde se observa la participación de los integrantes del grupo en la construcción, corrección y documentación de la práctica experimental.



Figura 12. Historial de commits del repositorio GitHub del equipo, evidenciando la participación de los integrantes en el desarrollo del módulo frontend.